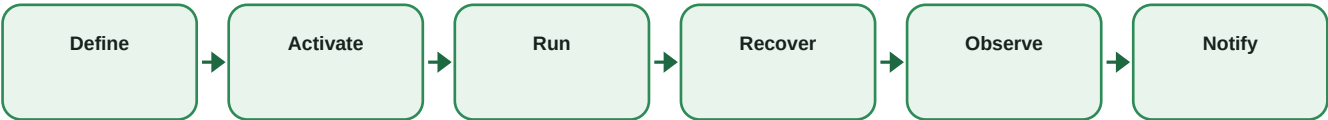


Packet Suite Integration Reality Check

What was implemented in PacketAgent, what contracts now exist for PacketADE and PacketChat, what was not changed in the other Packet applications, and what W10.3 PacketPhone work still means.

PacketAgent changed	Other app repos unchanged	W10.3 next
---------------------	---------------------------	------------



Pause state	Direct answer: no, this work did not modify all Packet applications. It implemented PacketAgent's receiving, delivery, and callback boundaries. External products still need their own client-side implementations and live interoperability certification.
--------------------	---

Prepared from committed repository state b259a4f on 2026-07-28.

Direct answer

The work is substantial, but it is concentrated in the PacketAgent repository.

No - we did not implement code into every Packet app.

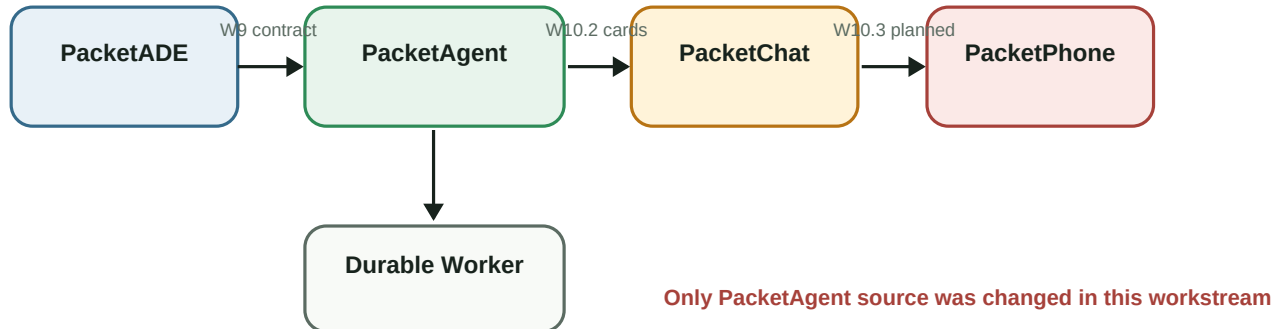
Product	Repository changed here?	Current integration truth
PacketAgent	Yes	Canonical Worker runtime, PacketADE receiver, PacketChat transport/callback server, outbox, controls, evidence, and APIs are implemented.
PacketADE	No	PacketAgent implements the receiving WorkerPackage/deployment/event contract and local fake-client tests. PacketADE UI/client wiring was not changed here.
PacketChat	No	PacketAgent can send the new message contract and authenticate callbacks. PacketChat UI/rendering/client code and a live endpoint were not changed or tested here.
PacketPhone	No	No Phone adapter or mutating callback is implemented. W10.3 is the next backlog loop.
PacketCode	No	No direct integration change was made. PacketCode remains the terminal coding surface in the suite model.

Why this matters

A server-side contract and adapter can be complete without the external product being integrated. W10.4 exists specifically to close that gap with local cross-adapter races and live endpoints.

Suite boundary model

PacketAgent owns durable runtime truth. Other Packet products are clients and notification/control surfaces.



Ownership by product

- **PacketADE** plans, builds, and supervises development work, then hands a WorkerPackage to PacketAgent.
- **PacketCode** is the terminal coding environment. No new runtime contract was added in this slice.
- **PacketChat** receives concise Worker cards and invokes read-only callbacks.
- **PacketPhone** is intended to carry remote approvals and kill controls after W10.3.
- **PacketAgent** owns the canonical Worker, durable scheduler, permission boundary, checkpoints, effects, evidence, budgets, and stop/revoke state.

Always on

The PacketAgent control plane stays available. Individual Worker runs are event-driven and bounded. No product should imply an endless autonomous token loop.

PacketADE integration: what exists and what does not

W9 is a complete PacketAgent-side receiver and reconnect contract, not proof that PacketADE shipped a matching button or client.

Implemented in PacketAgent

- Strict packetagent.worker-package/v1 parsing, canonical bytes, SHA-256 integrity, provenance, and optional DSSE verification.
- Workspace/actor/operation/expiry/revocation-bound PacketADE bearer credentials and local capability narrowing.
- Validate, deploy, update, activate, inspect, list runs, pause, resume, rollback, and revoke APIs.
- Reconnectable deployment/run event pages and bounded SSE with explicit durable acknowledgements.
- A serialized disconnect and fresh-service reconstruction gate through later update, pause/resume, rollback, and revoke.

Not implemented here

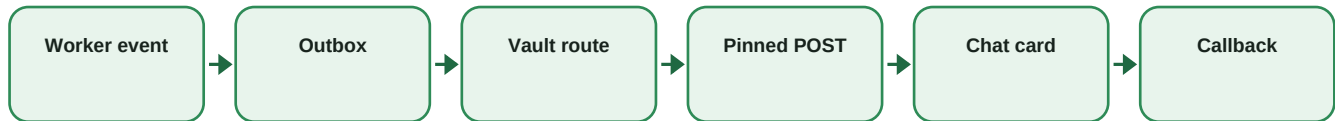
- PacketADE source code, UI, Deploy to PacketAgent button, credential setup, or production client.
- A real PacketADE endpoint-to-PacketAgent run in a deployed environment.
- Cross-repository version negotiation or release packaging.

Interpretation

PacketAgent is ready to receive the specified W9 contract. PacketADE must still adopt that contract and pass live interoperability before the suite can claim end-to-end deployment.

PacketChat W10.2: how the integration works

PacketAgent produces one channel-neutral outbox item, then the PacketChat adapter creates a bounded card and short-lived read-only callbacks.



Delivery path

- A Worker attention, progress, or terminal event writes evidence, an immutable envelope, an outbox item, and a retry job atomically.
- The scheduler claims the outbox item with a bounded attempt lease.
- The PacketChat adapter verifies the durable Worker/run/version/route binding before resolving the encrypted route.
- The adapter sends packetagent.packetchat-worker-message/v1 through the pinned-DNS network client with a stable idempotency key.
- Progress replaces one stable run card. Attention and terminal messages append to the run thread.

Card content

- Definition, deployment, run, version ID, version number/status, and immutable version digest.
- Deployment/run state and reason, budget policy and usage, latest checkpoint, evidence link, required action, title, and summary.
- Open and inspect callback URLs whose signed tokens are never persisted.

PacketChat W10.2: trust, setup, and limitations

The integration uses existing PacketAgent security primitives rather than a parallel secret or network model.

Operator setup inside PacketAgent

- Create an opaque packetagent.packetchat-route/v1 value through PUT /api/app/workers/credentials as a workspace admin.
- Declare that exact vault:* reference in the immutable Worker version and its packetchat notification route.
- The metadata-only credential API never returns plaintext or encrypted fields.

Callback authentication

- HS256 JWT binds PacketAgent issuer, PacketChat audience, action, exact Worker identity, version digest, route, issued time, expiry, and JTI.
- PacketAgent reloads durable state, resolves the exact route secret, verifies signature in constant time, and rejects tamper, expiry, or binding drift.
- open returns the matching workbench URL; inspect returns the W8 redacted read model. Both are no-store.

What PacketChat still needs

- An endpoint that accepts the message schema, honors Idempotency-Key, applies replace versus append semantics, and renders the callback affordances.
- A deployment credential and reachable HTTPS contract test.
- Live credential rotation and retry interoperability in W10.4.

No PacketChat repo changes

Nothing in this work proves the PacketChat application already understands this message. PacketAgent's adapter and fake-endpoint contract are complete; the receiving product still must implement and certify its side.

PacketPhone: W10.3 is next, not complete

The user referred to M10.3; the canonical backlog name is W10.3 PacketPhone controls.

Not implemented

- No PacketPhone outbound adapter is registered.
- No approve, reject, pause, stop, or revoke callback is exposed for Phone.
- No durable remote nonce-consumption record exists beyond W7's local control identities.
- No Phone actor/audience/role contract or live endpoint is configured.

W10.3 required design

- Deliver remote actions only when W7 permits the PacketPhone actor and role.
- Bind signed tokens to action, workspace, deployment, run, attention request, version digest, actor, audience, nonce, and expiry.
- Consume every mutating callback durably before returning success.
- Reject stale revision, replay, cross-workspace, cross-version, already-resolved attention, stopped runs, and revoked deployments.
- Route all mutations through the existing W7 control service so remote behavior cannot bypass local audit and policy.

Release consequence	The suite must not claim remote approvals or a unified PacketAgent-to-Phone lifecycle until W10.3 and W10.4 pass.
---------------------	---

What must happen in the other Packet products

These are external adoption steps, not hidden completed work.

Product	Adoption needed
PacketADE	Implement WorkerPackage v1 client, credential flow, deploy/keep-running UX, event reconnect and acknowledgement, update/rollback/control UX, and live W9 contract tests.
PacketChat	Implement message endpoint, schema validation, idempotent create/replace behavior, run threading, card rendering, open/inspect actions, error contract, and live W10.4 test endpoint.
PacketPhone	Agree actor/role/action schema, render controls, return signed callbacks, handle stale/replay outcomes, and participate in W10.3/W10.4 race tests.
PacketCode	No immediate action is required by W9-W10. A future handoff should be explicitly designed rather than inferred from its terminal role.
PacketAgent	Implement W10.3, run W10.4 local race/restart/rotation/dead-letter certification, then exercise all live endpoints before changing product claims.

Coordination rule	Do not silently create or guess another product's public contract. Version each interface, agree ownership, keep PacketAgent's runtime invariants, and test both sides against the same fixtures.
--------------------------	---

Pause handoff and decision summary

The repository is intentionally paused after W10.2.

What is committed

- b259a4f - PacketChat Worker delivery, callbacks, credential API, tests, and canonical documentation.
- All prior W1-W10.1 isolated commits remain on the branch history.
- The implementation gate is green: 1,493 API passes, 2 intentional skips, 28 web passes, successful typecheck/build, and zero lint errors.

Where to resume

- Open dev/CODEX-HANDOFF.md.
- Follow its direct link to BACKLOG.md W10.3.
- Treat BACKLOG.md as the sole active W10 and R1-R8 ledger.
- Use dev/worker-implementation-loops.md only for mechanics and verification detail.

Final status by product

Product boundary	Status
PacketAgent runtime	Implemented through W10.2
PacketADE -> PacketAgent	PacketAgent receiver complete locally; external product adoption/live test pending
PacketAgent -> PacketChat	PacketAgent sender/callback complete locally; PacketChat receiver/live test pending
PacketAgent <-> PacketPhone	Not implemented; W10.3 next
Unified suite claim	Not shipped until W10.4